

Final report - Step 9: Security hardening + full E2E validation

Project: amir-digital-store-v2 - Date: September 5, 2026

Mission: in-depth application security audit of the e-commerce journey (catalog -> cart -> checkout -> order -> Stripe -> webhook -> paid -> downloads -> admin). No new business features added.

1. Audit - vulnerabilities found

[V1][HIGH] server/routers.ts, server/db.ts

orders.createCheckoutSession({orderId}) with no capability: sequential, enumerable orderId. Anyone could attach a Stripe session to someone else's order (blocking the legitimate buyer) and push their email to Stripe. Fix: new orders.accessToken (256-bit hex, randomBytes), minted at creation, compared in constant time (isOrderTokenValid), required to open the Stripe session. Single generic error (no more existence oracle).

[V2][MEDIUM] server/db.ts

markOrderPaid/attachPaymentReference only guarded on paymentStatus: a late webhook could flip a cancelled/refunded order back to paid. Fix: added status = pending guard (idempotency preserved).

[V3][HIGH] server/downloads.ts

(a) Every success-page visit inserted new grants (unbounded live URLs). (b) The downloadCount < max check was verified then incremented in two queries (limit bypassable under concurrency). Fix: revoke previous grants per (order, asset) before minting; conditional atomic increment (UPDATE ... WHERE tokenHash AND expiresAt > now AND count < max, affectedRows verified).

[V4][MEDIUM] server/_core/index.ts

Unjustified 50mb JSON body limit (DoS). Fix: 1mb ; Stripe raw-body webhook preserved (route declared before the parser). + headers X-Content-Type-Options, X-Frame-Options: DENY, Referrer-Policy, X-Powered-By removed.

[V5][LOW] server/routers.ts, server/admin.ts

Admin errors returning raw error.message (DB internals could leak). Fix: PublicSafeError for safe business errors, safeAdminMessage() with generic fallback otherwise.

[V6][LOW] server/admin.ts

Admin search interpolated raw into LIKE (%/_ wildcards). Fix: escapeLikePattern().

[V7][LOW] server/storage.ts

resolve() without symlink resolution. Fix: realpath verification + prefix re-check in exists()/download().

[V8][INFO] server/payments.ts

async_payment_succeeded did not verify payment_status. Fix: paid required for both event types.

[V9][INFO] server/routers.ts

createCheckoutSession leaked 'Order not found' vs 'session already exists' (oracle). Fix: single generic message. No client-side secrets (VITE_* : IDs/URLs only).

Audited and found sound

- orders.create: prices/totals ignored, DB reload, unpublished rejected, quantities 1-99, at most 100 items, mixed-currency rejected.
- Session JWT (HS256, fixed alg), OAuth (state nonce + __Host- cookie, fixed / redirect), role from DB, cron forced to user.
- success_url never proof (Stripe API re-verification), download token (32 bytes, SHA-256 hash, expiry).
- FK restrict/cascade, .env ignored by Git.

2. Fixes - modified files

drizzle/schema.ts (+ migration drizzle/0005_lucky_firebird.sql: orders.accessToken), server/db.ts, server/payments.ts, server/downloads.ts, server/storage.ts, server/admin.ts, server/routers.ts, server/_core/index.ts, client/src/pages/Checkout.tsx (passes the token, in-memory only). No business features; Stripe/auth/Drizzle/tRPC architectures unchanged.

3. Security tests: 24 before -> 51 after, 51/51 PASSING

New: server/security.test.ts (17 tests: forged price/total ignored, extreme quantities, isOrderTokenValid, checkout token validation, empty/oversized tokens, ../absolute/backslash/symlink traversal, admin error hygiene, anti-setPaid scan) + server/webhook.test.ts (10 tests: signatures, redelivery, unpaid, metadata, verified paid).

4. Validation

- pnpm install --frozen-lockfile: PASS
- pnpm run check: PASS
- pnpm test: PASS (51/51)
- pnpm run build: PASS
- drizzle-kit check: PASS
- Built-server smoke test: anonymous admin.dashboard -> 403 PASS ; tokenless checkout -> generic 400 PASS ; headers present PASS.

5. Real-world testing

Actually tested: RBAC/IDOR (18 endpoints x anonymous/user, 403), Zod validation, webhook signatures (real HMAC verification), event routing, storage traversal, generic errors, HTTP headers, build.

Not tested: real MySQL, real Stripe, real webhook, real download, applied 0005 migration.

Impossible for lack of environment: empty DATABASE_URL, no STRIPE_*, no real PRIVATE_STORAGE_ROOT.

6. Remaining risks

- 1. Legacy orders (NULL accessToken) payable without token - transitional.
- 2. Customer name overwrite on repeat orders (inherent to guest checkout).
- 3. No rate limiting (order spam = DB fill).
- 4. VITE_FRONTEND_FORGE_API_KEY exposed by template design.
- 5. Project NOT declared production-ready: MySQL + Stripe test validation required before production.